

Java程序设计基础

Java Programming Fundamentals

Language Basics



Topics

- **Java 源代码基本框架**
- **标识符**
- **关键字**
- **数据类型**
- **基本输入输出**
- **运算符**

Java单文件程序基本框架

- 一个Java 源文件可包含多个类定义，但最多只能包含一个public类（主类）定义。
- 文件名与主类同名
- 要在一个主类中定义main方法
- PTA提交注意：
 - 默认的编译文件文件名为Main.java
 - 因此主类名必须为Main

```
//Filename Main.java
public class Main {
    public static void main (String args[]) {
        System.out.println("大家好!");
        System.out.println("Nice to meet you");
        Student stu = new Student();
        stu.speak("We are students");
    }
}

class Student {
    public void speak(String s) {
        System.out.println(s);
    }
}
```

Identifier & naming

- 定义
 - 用来标识类名、变量名、方法名、类型名、数组名、文件名的有效字符序列称为标识符。
- 标识符规定(Unicode)
 - 标识符由字母、下划线、美元符号和数字组成，长度不受限制。
 - 标识符的第一个字符不能是数字字符。
 - 标识符不能是关键字
 - 注意，采用unicode编码，汉字可以作为标识符

Key words / Reserved words

- <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/variables.html>
- 50个
- true false null

Key words

- 访问修饰符的关键字(3个)

关键字	意思	备注, 常用
public	公有的	可跨包, (默认选择)
protected	受保护的	当前包内可用
private	私有的	当前类可用

Key words

- 定义类、接口、抽象类和实现接口、继承类的关键字、实例化对象（共6个）

关键字	意思	备注，常用
class	类	<code>public class A(){} 花括号里有已实现方法体，类名需要与文件名相同</code>
interface	接口	<code>public interface B(){} 花括号里有方法体，但没有实现，方法体句子后面是英文分号“:”结尾</code>
abstract	声明抽象	<code>public abstract class C(){} 介于类与接口中间，可以有也可以没有已经实现的方法体</code>
implements	实现	用于类或接口实现接口 <code>public class A interface B(){} 实现接口</code>
extends	继承	用于类继承类 <code>public class A extends D(){} 继承类</code>
new	创建新对象	<code>A a=new A(); A表示一个类</code>

Key words

- 包的关键字 (共2个)

关键字	意思	备注, 常用
import	引入包的关键字	当使用某个包的一些类时, 仅需类名 然后使用ctrl+shift+o或者选定类名 (类或属性或方法) 按住ctrl+单击 即可自动插入类所在的包。如: JFrame 快捷键之后自动加入import javax.swing.JFrame;
package	定义包的关键字	将所有有关的类放在一个包类以便查找修改等。如: package javake.flycat.draw002;

Key words

- 数据类型的关键字 (共12个)

关键字	意思	备注, 常用
byte	字节型	8bit
char	字符型	16bit
boolean	布尔型	—
short	短整型	16bit
int	整型	32bit
float	浮点型	32bit
long	长整型	64bit
double	双精度	64bit
void	无返回	public void A(){} 其他需要返回的经常与return连用
null	空值	
true	真	
false	假	

Key words

- **流程控制 (共12个)**
 - if else while for switch case default do break
continue
 - return instanceof



Key words



• 修饰方法、类、属性和变量（共9个）

关键字	意思	备注，常用
static	静态的	属性和方法都可以用static修饰，直接使用类名.属性和方法名。只有内部类可以使用static关键字修饰，调用直接使用类名.内部类类名进行调用。static可以独立存在。静态块
final	最终的不可被改变的，用作常量	方法和类都可以用final来修饰、final修饰的类是不能被继承的、final修饰的方法是不能被子类重写，常量的定义：final修饰的属性就是常量
super	调用父类的方法	常见public void paint(Graphics g){super.paint(g); ...}
this	当前类的父类的对象	调用当前类中的方法（表示调用这个方法的对象）this.addActionListener(al)
native	本地	
strictfp	严格,精准	
synchronized	线程,同步	
transient	短暂	
volatile	易失	

Key words

• 包的关键字（共2个）

关键字	意思	备注，常用
import	引入包的关键字	当使用某个包的一些类时，仅需类名 然后使用ctrl+shift+o或者选定类名（类或属性或方法）按住ctrl+单击 即可自动插入类所在的包。如：JFrame 快捷键之后自动加入import javax.swing.JFrame;
package	定义包的关键字	将所有有关的类放在一个包类以便查找修改等。如：package javake.flycat.draw002;

Key words

- 错误处理 (共5个)
 - try catch finally throw throws

Primitive Data Types

- 有8种基本数据类型：
 - boolean、byte、short、int、long、float、double、char
- 分为四类

逻辑类型

boolean

整数类型

byte、short、
int、long

字符类型

char

浮点类型

float、
double

Boolean

变量

使用关键字boolean来
声明逻辑变量，声明时
也可以赋给初值。

例如：

```
boolean x, ok=true, 关闭=false;
```

常量

true、false

整型

int

变量：使用关键字int来声明int型变量,声明时也可以赋给初值.

例如: `int x= 12,平均=9898,jiafei;`

常量：int型常量共有4种表示方法:

- 十进制: 123, 6000 (十进制)
- 八进制: 077 (八进制, 数字零做前缀)
- 十六进制: 0x3ABC (十六进制, 0x或 0X做前缀)
- 二进制: 0b111(用0b或0B做前缀)。

对于int型变量, 内存分配给4个字节 (byte) ,占32位.

byte

变量：使用关键字byte来声明byte 型变量.

例如: `byte x= -12, tom=28, 漂亮=98;`

常量：一定范围内的int型常量赋值给byte型变量.

对于byte型内存分配给1个字节, 占8位 .

整型

short 型

变量：使用关键字short来声明short型变量。

例如： `short x=12,y=1234;`

常量：和byte型类似，Java中也不存在short型常量的表示法，但可以把一定范围内的int型常量赋值给short型变量。

对于short型变量，内存分配给2个字节，占16位。

long 型

变量：使用关键字long来声明long型变量。

常量：long型常量用后缀L来表示

例如108L(十进制)、07123L(八进制)、0x3ABCL(十六进制)。

例如： `long width=12L,height=2005L,length;`

对于long型变量，内存分配给8个字节，占64位。

字符型char

- 常量: 'A' , 'b' , '?' , '!' , '9' , '好' , '\t' , 'き' , '毛' 等, 即用单引号扩起的Unicode表中的一个字符。

- 变量: 使用关键字char来声明char型变量, 对于char型变量, 内存分配给2个字节, 占16位

例如: `char ch= 'A' ,home= '家' ,handsome= '酷' ;`

- 转意字符常量:

有些字符 (如回车符) 不能通过键盘输入到字符串或程序中, 就**需要**使用转意字符常量,

例如: \n(换行), \b(退格), \t(水平制表), \ ' (单引号),

\ " (双引号), \\ (反斜线)等。

- Java语言使用Unicode标准字符集, 最多可以识别65536个字符。

常用转义字符

- `\b` (backspace)
- `\t` (tab)
- `\n` (line feed)
- `\f` (form feed)
- `\r` (carriage return)
- `\"` (double quote)
- `\'` (single quote)
- `\\` (backslash).

浮点型-float

- 常量: 453.54F(小数表示法), 2e40f(2乘10的40次方, 指数表示法).
- 变量: 使用关键字float来声明float型变量,
例如: `float x=22.76f,tom=1234.987f,weight=1e-12F;`
- 精度:float变量在存储时保留8位有效数字。
- 对于float型变量, 内存分配给4个字节, 占32位。

需要特别注意的是float常量后面必须要有后缀f或F。

浮点型-double

- **常量**: 238.539d, 231.987(小数表示法), 1e-90(1乘10的-90次方, 指数表示法).对于double常量, 后缀有 “d” 或 “D” , **但允许省略后缀**.
- **变量**: 使用关键字double来声明double型变量,
例如: `double height=23.345,width=34.56D,length=1e12;`
- 对于double型变量, 内存分配给8个字节, 占64位。
- **精度**:double变量在存储double型数据时保留16位有效数字, 实际精度取决于具体数值。

特别注意

一个具有小数部分的数据的缺省类型是double而不是float。所以float常量后面必须要有后缀 “f” 或 “F” 。



Using Underscore Characters in Numeric Literals

- `long creditCardNumber = 1234_5678_9012_3456L;`
- `long socialSecurityNumber = 999_99_9999L;`
- `float pi = 3.14_15F;`
- `long hexBytes = 0xFF_EC_DE_5E;`
- `long hexWords = 0xCAFE_BABE;`
- `long maxLong = 0x7fff_ffff_ffff_ffffL;`
- `byte nybbles = 0b0010_0101;`
- `long bytes = 0b11010010_01101001_10010100_10010010;`

Data Type Conversion

- 按精度从“低”到“高”排列：
 - byte short char int long float double
- 转换规则
 - 当把级别低的变量的值赋给级别高的变量时，系统自动完成数据类型的转换。
 - 例如：float x=100;
 - 当把级别高的变量的值赋给级别低的变量时，必须使用显示类型转换运算。显示转换的格式：（类型名）要转换的值；
 - 例如：int x=(int)34.89;
 - 当把一个int型常量赋值给一个byte和short型变量时，不可以超出这些变量的取值范围，否则必须进行类型转换运算；
 - 例如： byte b=128;(error) byte b=(byte)128(ok)

Default Values

Data Type	Default Value (for fields)
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0d
char	'\u0000'
String (or any object)	null
boolean	false

- 注意:
 - 局部变量不会自动赋缺省值

String

- **String 常量**

- 用双引号括起的字符序列，例如：“你好”、“12.97”、“boy”等。

- **String对象**

- `String s = new String( "NEUQer");`
 - `String s = "NEUQer" ;`

- `char a[] = {'J','a','v','a'};`
 - `String s = new String(a);`

- **求string的长度**

- `s.length();`

- **字符串拼接: +**

- `string1.concat(string2);`

BigInteger

- **java.math.BigInteger**用来表示任意大小的整数。
 - **BigInteger**内部用一个int[]数组来模拟一个非常大的整数
 - **构造函数**

方法	描述
BigInteger(int bitLength, int certainty, Random rnd)	构造一个随机生成的正 BigInteger，它可能是一个具有指定 bitLength 的素数。
BigInteger(int numBits, Random rnd)	构造一个随机生成的 BigInteger，它是在 0 到 (2numBits - 1) (包括) 范围内均匀分布的值。
BigInteger(byte[] val)	将包含 BigInteger 的二进制补码表示形式的 byte 数组转换为 BigInteger
BigInteger(int signum, byte[] magnitude)	将 BigInteger 的符号-数量表示形式转换为 BigInteger
BigInteger(String val)	将BigInteger 的十进制字符串表示形式转换为 BigInteger
BigInteger(String val, int radix)	将指定基数的 BigInteger 的字符串表示形式转换为 BigInteger

BigInteger常用函数

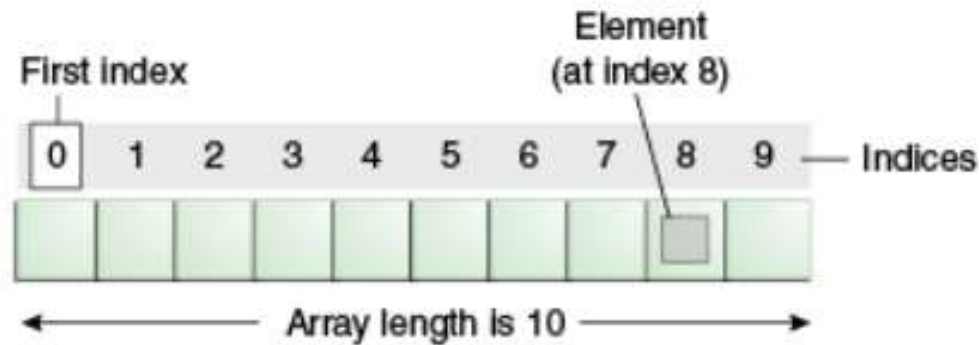
方法	描述
<code>add(BigInteger val)</code>	加法: 返回一个BigInteger值与另一个BigInteger值相加后的BigInteger值,
<code>subtract(BigInteger val)</code>	减法: 返回其值为 $(this - val)$ 的 BigInteger
<code>multiply(BigInteger val)</code>	乘法: 返回其值为 $(this * val)$ 的 BigInteger
<code>divide(BigInteger val)</code>	除法: 求除数,不带小数值,返回其值为 $(this / val)$ 的 BigInteger 可能会产生除零异常
<code>remainder(BigInteger val)</code>	求余数,返回其值为 $(this \% val)$ 的BigInteger
<code>and(BigInteger val)</code>	返回其值为 $(this \& val)$ 的 BigInteger
<code>abs()</code>	返回BigInteger绝对值表示形式
<code>compareTo(BigInteger val)</code>	BigInteger与val比较大小,当此 BigInteger 在数值上小于、等于或大于 val 时, 返回 -1, 0, 或 1。
<code>pow(BigInteger val)</code>	此 BigInteger 的指数,返回 $this ^ val$ 的BigInteger
<code>shiftLeft(BigInteger val)</code>	左移 :返回其值为 $(this \ll n)$ 的 BigInteger。
<code>shiftRight(BigInteger val)</code>	右移: 返回其值为 $(this \gg n)$ 的 BigInteger。
<code>toString()</code>	返回此 BigInteger 的十进制字符串表示形式
<code>toString(int radix)</code>	返回此 BigInteger 的给定基数的字符串表示形式。
<code>intValue()</code>	将此 BigInteger 转换为int
<code>longValue()</code>	将此 BigInteger 转换为 long

BigDecimal

- 解决float和double运算精度问题
 - `public BigDecimal(String val)` 将字符串表示形式转换为BigDecimal
- 运算
 - 加法 `public BigDecimal add(BigDecimal val)`
 - 减法 `public BigDecimal subtract(BigDecimal val)`
 - 乘法 `public BigDecimal multiplyt(BigDecimal val)`
 - 除法 `public BigDecimal divide(BigDecimal divisor,int scale,int roundingMode)`
 - `int scale`代表精度,`int roundingMode`代表舍入模式 比如: 四舍五入

Arrays

An *array* is a container object that holds a fixed number of values of a single type. The length of an array is established when the array is created. After creation, its length is fixed.



An array of 10 elements.



Arrays



Declaring a Variable to Refer to an Array

```
// declares an array of integers  
int[] anArray;
```

```
// this form is discouraged  
float anArrayOfFloats[];
```

Arrays

Creating and Initializing

```
// create an array of integers
anArray = new int[10];
anArray[0] = 100; // initialize first element
anArray[1] = 200; // initialize second element
anArray[2] = 300; // and so forth

int[] anArray = {
    100, 200, 300,
    400, 500, 600,
    700, 800, 900, 1000
};
```

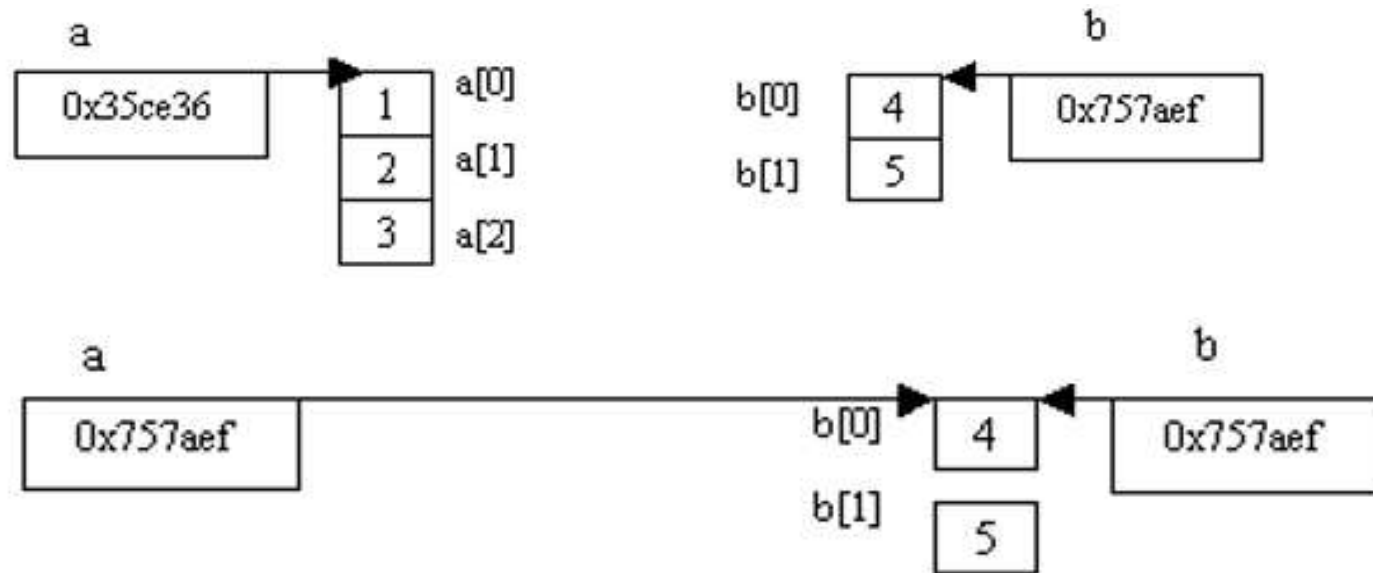
You can also declare an array of arrays (also known as a *multidimensional array*)

> Arrays

Variable of array is a reference:

```
int a[] = {1, 2, 3}, b[] = {4, 5};
```

```
a=b
```





Java.util.Arrays



```
class ArrayCopyOfDemo {  
    public static void main(String[] args) {  
        String[] copyFrom = {  
            "Affogato", "Americano", "Cappuccino", "Corretto", "Cortado",  
            "Doppio", "Espresso", "Frappucino", "Freddo", "Lungo", "Macchiato",  
            "Marocchino", "Ristretto" };  
  
        String[] copyTo = java.util.Arrays.copyOfRange(copyFrom, 2, 9);  
        for (String coffee : copyTo) {  
            System.out.print(coffee + " ");  
        }  
    }  
}
```

Input / Output

- **How to read from console**
- **How to write to console**

How to read from console

- **Scanner**

- `import java.util.Scanner;`
- 读入int: `nextInt()`
- 读入short: `nextShort()`
- 读入long: `nextLong()`
- 读入float: `nextFloat()`
- 读入double: `nextDouble()`
- 读入byte: `nextByte()`
- 读入boolean: `nextBoolean()`

```
Scanner reader=new Scanner(System.in);  
int i=reader.nextInt();
```

- 读入字符串String: `next()`
- 读入整行(含空格): `nextLine()`
- 读入一个字符char
 - 作为一个字符串读入
 - 取出第0个字符

```
String str=reader.next();  
char ch = str.charAt(0);
```

How to write to console

- **System.out.write – 不建议使用**
 - `System.out.write(int b);`
 - `System.out.write(byte[] buf, int off, int len)`
 - 写入缓冲区，调用`System.out.flush()`会输出到console
- **一般可以使用经过封装后的print和println**
 - `System.out.print( "hello" + "world" + 123);`
 - `System.out.println( "hello" + "world" + 123);`
 - 可以输出任意类型（基础类型，自定义类型会自动输入`toString()`的结果）

How to write to console

- 复杂格式输出：System.out.printf和System.out.format
 - 参考C语言中的printf()函数

```
1 public class Main{
2     public static void main(String[]args){
3         double d = 345.678;
4         String s = "hello!";
5         int i = 1234;
6         //%"表示进行格式化输出，%"之后的内容为格式的定义。
7         System.out.printf("%f\n",d); //"f"表示格式化输出浮点数。
8         System.out.printf("%9.2f\n",d); //"9.2"中的9表示输出的长度，2表示小数点后的位数。
9         System.out.printf("%+9.2f\n",d); //"+"表示输出的数带正负号。
10        System.out.printf("%-9.4f\n",d); //"-"表示输出的数左对齐(默认为右对齐)。
11        System.out.printf("%+-9.3f\n",d); //"+-"表示输出的数带正负号且左对齐。
12        System.out.printf("%d\n",i); //"d"表示输出十进制整数。
13        System.out.printf("%o\n",i); //"o"表示输出八进制整数。
14        System.out.printf("%x\n",i); //"d"表示输出十六进制整数。
15        System.out.printf("%#x\n",i); //"d"表示输出带有十六进制标志的整数。
16        System.out.printf("%s\n",s); //"d"表示输出字符串。
17        System.out.printf("输出一个浮点数: %f, 一个整数: %d, 一个字符串: %s\n",d,i,s); //可以输出多个变量，注意顺序。
18        System.out.printf("字符串: %2$s, %1$d的十六进制数: %1$#x\n",i,s); //"X$"表示第几个变量。
19    }
20 }
```

```
345.678000
 345.68
+345.68
345.6780
+345.678
1234
2322
4d2
0x4d2
hello!
输出一个浮点数: 345.678000, 一个整数: 1234, 一个字符串: hello!
字符串: hello!, 1234的十六进制数: 0x4d2
```

日志和调试输出

- 程序调试/过程数据输出
- `System.out.print`
- 错误信息输出
- `System.err.print`



Operators

运算符	描述	结合性
()	圆括号，用于改变运算顺序	从左到右
[]	数组索引访问	从左到右
.	成员访问操作符	从左到右
!	逻辑非（取反）	从右到左
~	按位取反	从右到左
++, --	前缀递增和递减，后缀递增和递减	从右到左
*, /, %	乘法，除法，取模	从左到右
+, -	加法，减法	从左到右
<<, >>, >>>	左移，右移，无符号右移	从左到右
<, <=, >, >=	小于，小于等于，大于，大于等于	从左到右
instanceof	检查对象是否是指定类的实例	从左到右
==, !=	相等，不相等	从左到右
&	按位与	从左到右
^	按位异或	从左到右
	按位或	从左到右
&&	逻辑与	从左到右
	逻辑或	从左到右
?:	条件运算符	从右到左
=, +=, -=, *=, /=, %=	赋值，复合赋值运算符	从右到左
&=, ^=, =, <<=, >>=, >>>=	位运算符与赋值运算符	从右到左